The Lunar Heat-Flow Study: A Guidebook from the Ground Up

The physics, the code, and how the two fit together

A plain-language companion to "Difference of Lunar Regolith Thermal Conductivity K_d at the Apollo 15 and 17 Heat-Flow Boreholes"

June 15, 2026

How to use this book

You need *no* background in physics or programming. Every topic gives the **physics in plain words first, then the actual code that performs it**, walked through line by line. The coloured boxes guide you: **plain-words** restate an idea simply; **equation** boxes name every symbol; **code** boxes connect a formula to the lines that compute it; **key-takeaway** boxes say what to remember. The code is copied from the repository you can download and run yourself.

Contents

1 Why anyone studies heat in the Moon **2**

2 Heat, temperature, and the law of conduction **2**
2.1 What heat and temperature actually are . . . 3
2.2 Conduction . . . 3
2.3 Two words we need: flux and gradient . . . 3
2.4 Fourier's law: the rate of heat flow . . . 3

3 The materials: turning physics into numbers and code **4**
3.1 Conductivity that grows with depth and temperature . . . 4
3.2 Density and specific heat: how much heat the soil stores . . . 4
3.3 Where the numbers come from . . . 5

4 The heat equation: bookkeeping for energy **5**

5 The lunar column: forcing, skin, and the steady zone **6**
5.1 Heated from both ends . . . 6
5.2 The skin depth: why the daily swing dies out . . . 6
5.3 The steady deep zone and the gradient law . . . 7

6 Step 1 — the depth grid **7**
6.1 Physics: put the detail where the action is . . . 7
6.2 Code: building the grid . . . 7

7 Step 2 — from calculus to arithmetic **8**
7.1 Method: differences instead of derivatives . . . 8
7.2 Conductivity at the boundary between two layers . . . 8
7.3 Stepping forward in time: Crank–Nicolson . . . 8
7.4 Code: assembling one time step . . . 9

8 Step 3 — solving the layer equations fast **9**

9 Step 4 — the surface: balancing sunlight and glow **10**
9.1 Physics: the surface energy balance . . . 10
9.2 Method: Newton's method finds T_s . . . 10

10 Step 5 — the bottom: the geothermal flux **10**

11 Step 6 — settling to equilibrium **11**
11.1 Physics: why settling is slow . . . 11
11.2 Method and code: the shortcut . . . 11

12 A worked trace: watching the machine run **11**
12.1 One time step, a layer at 0.5 m, around 252 K . . . 11
12.2 One full retrieval, Apollo 15 . . . 12

13 The Apollo data and how it is cleaned **12**

14 The retrieval: guess, simulate, compare **12**

15 Uncertainty I: the bootstrap **13**

16 Uncertainty II: Bayesian inference and MCMC **14**

17 Choosing between explanations: the AICc **15**

18 The whole pipeline at a glance **15**

19 The mistake we found and fixed **17**
19.1 The bug . . . 17
19.2 The fix and the check . . . 17
19.3 What changed . . . 17

20 The results and what they mean **17**

21 Honest limits and the bottom line **18**

22 How to run the code yourself **20**

23 Glossary of every term **20**

PART I • The Physics, and the Code That Implements It

1 Why anyone studies heat in the Moon

The Moon looks dead and unchanging, but just beneath its surface there is a slow, quiet flow of heat that carries information we can get no other way. Its surface is roasted by the Sun through the two-week lunar day and freezes through the two-week night, swinging by roughly 250 °C. Yet a single metre down, none of that drama is felt: there the temperature is set by only two things — how well the soil conducts heat, and a faint steady warmth seeping up from the Moon's interior.

The key property of the soil is its **thermal conductivity** — how easily heat passes through it. We write it K, and its value deep down (where the soil is compacted and settled) is K_d. Lunar soil is an extraordinary insulator — several times better than the styrofoam in a coffee cup — which is exactly why surface heat cannot reach far below.

Why it matters: every lunar thermal model today uses *one single value of K_d for the whole Moon*, never checked against direct underground data — because such data exist at only two places in history, the Apollo 15 and 17 landing sites, where astronauts buried thermometers in 1971–72.

Key takeaway

We use the only two underground temperature records ever taken on the Moon to test a hidden, universal assumption: is one conductivity value good enough for the entire Moon? We will build, step by step, to the answer — no: the two sites differ by about a factor of two, and the difference is real.

2 Heat, temperature, and the law of conduction

2.1 What heat and temperature actually are

Everything is made of atoms, and atoms are always jiggling. **Temperature** measures the *average* jiggle: hot means vigorous jiggling, cold means sluggish. **Heat** is the *energy* of that jiggling on the move — energy flowing from a more-jiggly place to a less-jiggly one. Temperature is measured in kelvin (K): 0 K is no jiggling at all (about −273 °C), water freezes at 273 K, and lunar deep soil sits near 250 K (−23 °C).

2.2 Conduction

When a hot region touches a cold region, the fast atoms jostle their slower neighbours, passing energy along (Figure 1). No material moves — only energy does, handed atom to atom. It is how a metal spoon's handle warms in hot soup, and it is the *only* way heat travels inside airless lunar soil (no air to carry it, and the gaps are too small for much convection).

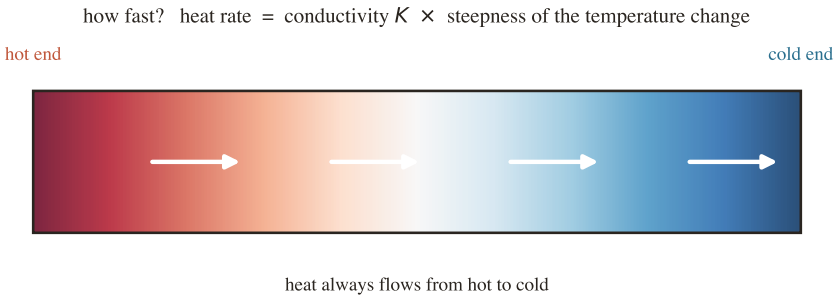


Figure 1. Conduction. Heat always flows from hot to cold. How fast depends on the conductivity K and on how steep the temperature change is — a steeper change, or a better conductor, moves more heat per second.

2.3 Two words we need: flux and gradient

Heat flux (q) is how much heat energy crosses one square metre each second, in watts per square metre ($\mathrm{W\,m^{-2}}$). **Temperature gradient** is how fast temperature changes with distance — here, with depth z. If the temperature rises 2 degrees over half a metre, the gradient is 4 K per metre. We write it $\partial T/\partial z$, which is just shorthand for "the slope of the temperature-versus-depth line."

2.4 Fourier's law: the rate of heat flow

Two centuries ago Joseph Fourier wrote the rule connecting these: heat flux equals conductivity times gradient.

The equation, term by term

$$q \;=\; -\,K\,\frac{\partial T}{\partial z}$$

q — heat flux ($\mathrm{W\,m^{-2}}$). K — conductivity ($\mathrm{W\,m^{-1}\,K^{-1}}$); a big K passes heat easily. $\partial T/\partial z$ — the temperature gradient ($\mathrm{K\,m^{-1}}$). The minus sign means heat flows *down* the gradient, from hot toward cold.

A worked number. Deep lunar soil conducts about $K \approx 0.009\,\mathrm{W\,m^{-1}\,K^{-1}}$ and its temperature rises about 1.8 K per metre at Apollo 17. So the upward heat flux is $q = 0.009 \times 1.8 \approx 0.016\,\mathrm{W\,m^{-2}}$ — about a hundred-thousandth of the noon sunlight ($\sim$1360 $\mathrm{W\,m^{-2}}$). That tiny trickle is the basal heat flux we will keep meeting.

In plain words

A crowded room empties through a door faster if the door is wide (big K) and if it is far more crowded inside than out (steep gradient). Heat is the crowd; Fourier's law is the turnstile count.

3 The materials: turning physics into numbers and code

Fourier's law needs the conductivity K; once things change in time we also need density ρ and specific heat c_p. This is our first "physics, then code" pairing.

3.1 Conductivity that grows with depth and temperature

Two physical facts shape lunar conductivity. **Depth:** surface dust is fluffy and loosely packed (a poor conductor); deeper down it is compressed, so grains touch better and conduct better — K rises from a small surface value K_s to a larger deep value K_d over a short transition depth H. **Temperature:** heat also crosses the tiny gaps between grains as infrared light (radiation), which grows fast with temperature (as T^3), so warm soil conducts a bit better — the "radiative term," controlled by a coefficient χ.

The equation, term by term

$$K(T,z) = \underbrace{\left[\, K_d - (K_d - K_s)\, e^{-z/H} \,\right]}_{\text{grows with depth}} \times \underbrace{\left[\, 1 + \chi\,(T/350)^3 \,\right]}_{\text{grows with temperature}}$$

First bracket: starts at K_s at the surface ($z = 0$, where $e^0 = 1$ so the two K_d terms leave K_s) and climbs toward K_d deep down (where $e^{-z/H} \to 0$). Second bracket: 1 plus the radiative boost; χ sets its strength, 350 K is a reference temperature. K_d is the single number we retrieve.

Worked numbers, with the real constants ($K_s = 7.4 \times 10^{-4}$, $K_d = 3.4 \times 10^{-3}\,\mathrm{W\,m^{-1}\,K^{-1}}$, $H = 0.06\,\mathrm{m}$, $\chi = 2.7$): at the surface the depth bracket is just K_s; by $z = 0.2\,\mathrm{m}$, $e^{-0.2/0.06} = e^{-3.3} \approx 0.036$, so it has reached ∼97% of K_d. The radiative bracket at 250 K is $1 + 2.7 \times (250/350)^3 = 1 + 2.7 \times 0.36 \approx 2.0$ — it *doubles* the conductivity, which is why forgetting this term is a known, serious bug.

In plain words

Conductivity is not one number but a recipe: "start small at the dusty top, rise to K_d once packed, and add a little extra when warm." Our job is to find the one ingredient K_d that makes this recipe match the buried thermometers.

Here is that formula in code — each line matches a bracket above.

`file: lunar/properties.py`

```
def conductivity_hayne(T, z, Ks=K_SURFACE, Kd=K_DEEP,
                       H=H_PARAMETER, chi=CHI_RADIATIVE):
    T = np.asarray(T, dtype=np.float64)
    z = np.asarray(z, dtype=np.float64)
    Kc = Kd - (Kd - Ks) * np.exp(-z / H)            # grows with depth
    return Kc * (1.0 + chi * (T / T_REFERENCE)**3)  # times radiative boost
```

How the code does it

`Kc = Kd - (Kd - Ks)*np.exp(-z/H)` is the first bracket exactly (`np.exp(-z/H)` is $e^{-z/H}$). The return multiplies by the radiative bracket `(1.0 + chi*(T/T_REFERENCE)**3)`, where `**3` is "cubed" and `T_REFERENCE`= 350. Wrapping `T` and `z` in `np.asarray` lets NumPy compute every depth at once, in one shot, instead of looping.

3.2 Density and specific heat: how much heat the soil stores

When temperatures change in time we need how much energy it takes to warm the soil: its **density** ρ (mass per cubic metre) and **specific heat** c_p (energy to warm one kilogram by one degree). Density rises with depth as the soil compacts, with the same exponential shape as the conductivity:

The equation, term by term

$$\rho(z) = \rho_d - (\rho_d - \rho_s)\, e^{-z/H}$$

Surface density $\rho_s \approx 1100$ climbing to deep density $\rho_d \approx 1800\,\mathrm{kg\,m^{-3}}$ over the same depth scale H.

file: lunar/properties.py

```
def density_hayne(z, rho_s=RHO_SURFACE, rho_d=RHO_DEEP, H=H_PARAMETER):
    z = np.asarray(z, dtype=np.float64)
    return rho_d - (rho_d - rho_s) * np.exp(-z / H)
```

Specific heat depends on temperature (cold soil stores heat differently from warm soil) and is a polynomial fit to laboratory measurements of Apollo samples:

file: lunar/properties.py

```
def _cp_hayne(T):                    # c_p(T): a 4th-order polynomial
    T = np.asarray(T, dtype=np.float64)
    return (CP_HAYNE_C0 + CP_HAYNE_C1*T + CP_HAYNE_C2*T**2
            + CP_HAYNE_C3*T**3 + CP_HAYNE_C4*T**4)
```

How the code does it

Just $c_0 + c_1T + c_2T^2 + c_3T^3 + c_4T^4$ — five measured constants times powers of temperature, added up. Near 250 K it gives $c_p \approx 670\,\mathrm{J\,kg^{-1}\,K^{-1}}$ (about a sixth of water's): it takes 670 joules to warm one kilogram of regolith by one degree.

3.3 Where the numbers come from

A rule of this project: *no number is invented.* Every constant lives in one file with its literature source attached.

file: lunar/constants.py

```
#: Surface contact conductivity [W m^-1 K^-1] -- Hayne et al. (2017).
K_SURFACE: float = 7.4e-4
#: Deep contact conductivity [W m^-1 K^-1] -- Hayne et al. (2017).
K_DEEP: float = 3.4e-3           # the global value we test against
#: H-parameter (transition depth scale) [m] -- Hayne et al. (2017).
H_PARAMETER: float = 0.06
#: Stefan-Boltzmann constant [W m^-2 K^-4] -- CODATA 2018.
SIGMA_SB: float = 5.670374419e-8
```

Key takeaway

The physics enters the computer as three small functions — `conductivity_hayne`, `density_hayne`, `specific_heat` — each a direct transcription of a formula, fed by cited constants. Understand the three formulas and you understand every material input to the model.

4 The heat equation: bookkeeping for energy

Fourier's law gives the flow through one surface. To follow temperature *everywhere, over time*, we add one idea: **energy bookkeeping**. Take a thin slice of soil. In one moment, some heat flows in through its top face and some flows out the bottom face (each given by Fourier's law). If more comes in than leaves, the leftover energy stays and warms the slice; if more leaves, it cools. Write that balance down, and you get the **heat equation** — the master equation the whole program solves.

The equation, term by term

$$\underbrace{\rho\, c_p \frac{\partial T}{\partial t}}_{\text{rate heat is stored in the slice}} = \underbrace{\frac{\partial}{\partial z}\left(K \frac{\partial T}{\partial z}\right)}_{\text{(heat flowing in)}-\text{(heat flowing out)}}$$

Left: how fast the slice warms ($\partial T/\partial t$, degrees per second) times how much heat that costs (ρc_p, the slice's "thermal inertia"). **Right:** the net conduction — Fourier's flux $K\,\partial T/\partial z$ differenced across the slice (the outer $\partial/\partial z$ asks "how much more flux leaves than enters"). If the right side is zero, in equals out and the slice holds steady.

In plain words

"Temperature here changes because more heat arrives than leaves." That one sentence *is* the heat equation. Everything the computer does is apply it to thousands of thin slices, over and over, as the lunar day passes.

This equation cannot be solved with pencil and paper for a real, property-varying column, so we solve it numerically — the whole of Part II.

5 The lunar column: forcing, skin, and the steady zone

5.1 Heated from both ends

Our column of soil (Figure 2a) is driven from *above* by sunlight (and night-time cooling) and from *below* by a steady warmth from the interior — the **basal heat flux** Q_b, a few thousandths of a watt per square metre (21 at Apollo 15, 15 at Apollo 17, in milliwatts per square metre).

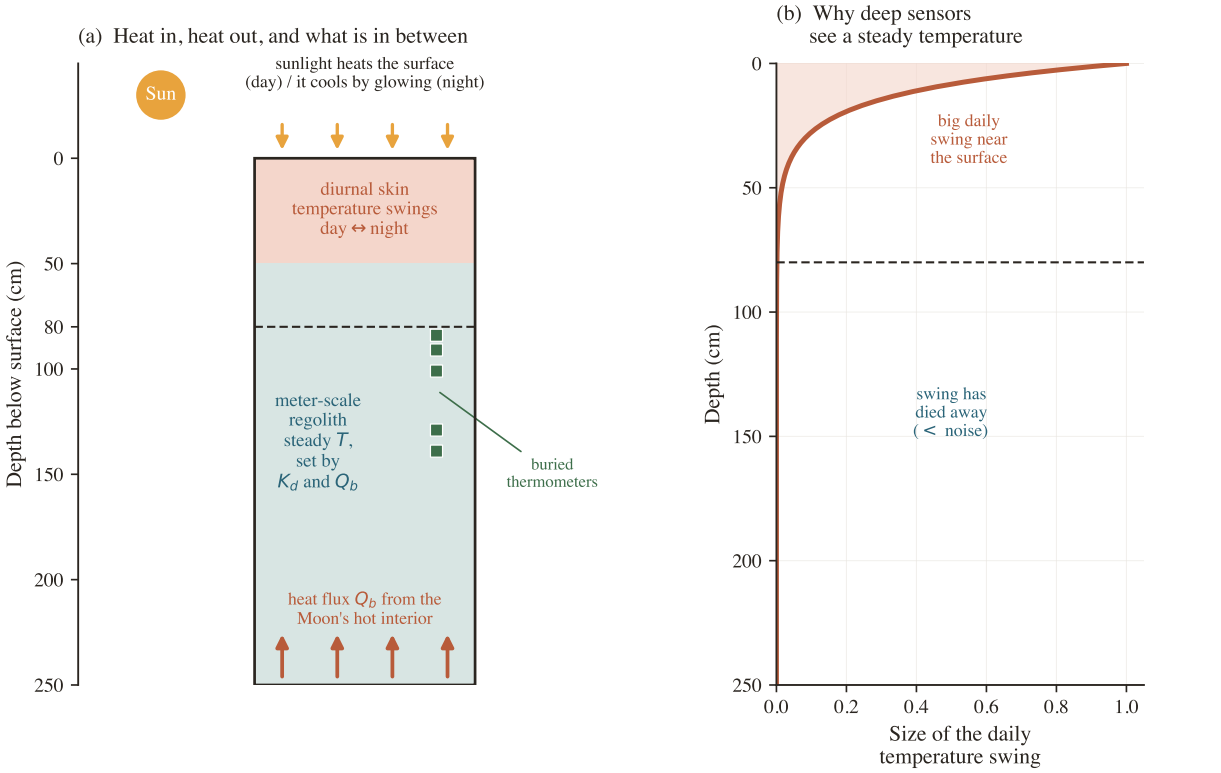


Figure 2. The lunar soil column. (a) Sunlight heats from above; Q_b rises from below. Top zone (coral): the day–night swing. Deep zone (teal): steady. Squares: the buried thermometers. (b) The daily swing collapses with depth — below the noise by 80 cm, which is why deep sensors read a steady value.

5.2 The skin depth: why the daily swing dies out

The day–night wave is absorbed and smeared by each layer it passes, so it shrinks fast with depth (Figure 2b). The depth over which it fades is the **skin depth**, $\delta = \sqrt{2\kappa/\omega}$, where $\kappa = K/(\rho c_p)$ is the

"thermal diffusivity" and ω is the day–night rhythm. For lunar soil δ is only a few centimetres; the swing shrinks by a factor $e \approx 2.7$ every skin depth, so by 80 cm (more than ten skin depths) it has shrunk by $e^{-10} \approx 1/22000$ — far below what any thermometer can read.

In plain words

Press a warm hand on a thick quilt: the surface warms, a finger-width in, nothing. The Moon's daily heating is that hand — it reaches only the top few centimetres. Everything deeper feels only the steady average.

5.3 The steady deep zone and the gradient law

Below the skin depth the soil is in **steady state**: its temperature no longer wobbles, only climbs slowly with depth as Q_b trickles up. There the time term of the heat equation vanishes and it collapses to the simple law that lets us measure K_d:

The equation, term by term

$$\frac{\Delta T}{\Delta z} = \frac{Q_b}{K_d} \qquad \Longrightarrow \qquad K_d = \frac{Q_b}{\Delta T/\Delta z}$$

The deep temperature climbs at a steady rate equal to the flux Q_b divided by the conductivity K_d. We *measure* the gradient from the buried thermometers and *know* Q_b, so we solve for K_d.

Key takeaway

This is the whole study in one line: read the deep temperature gradient, divide the known heat flux by it, get the conductivity. Everything else is doing this carefully — with the full time-dependent model instead of the shortcut, and with honest error bars.

PART II • How the Computer Solves the Equation

6 Step 1 — the depth grid

6.1 Physics: put the detail where the action is

All the fast change happens near the surface (the skin). So instead of slicing the column into equal layers, we make the top layers extremely thin (2 mm) and let each successive layer be a little thicker. This spends computing effort where temperature changes fastest and saves it deep down where nothing moves. (A uniform grid is forbidden in this project for exactly this reason — it would either miss the skin or waste effort.) The production column is 5 m deep and ends up with a few hundred layers.

6.2 Code: building the grid

`file: lunar/grid.py`

```
def make_geometric_grid(z_max=3.0, dz0=0.002, growth=0.15):
    faces = [0.0]
    dz = dz0
    while faces[-1] < z_max:
        faces.append(faces[-1] + dz)    # next layer boundary
        dz *= 1.0 + growth              # each layer thicker by 'growth'
    z_face = np.asarray(faces)
    dz_arr = np.diff(z_face)                     # layer thicknesses
    z_mid  = 0.5 * (z_face[:-1] + z_face[1:])  # layer centres
    return DepthGrid(z_face=z_face, z_mid=z_mid, dz=dz_arr)
```

How the code does it

Start at the surface with a 2 mm layer (`dz0=0.002`). Each loop adds the next boundary, then grows the thickness by a few percent (`dz *= 1+growth`), so layers gently fatten until the column reaches `z_max`. `z_face` are the boundaries, `z_mid` the centres (where temperature lives), `dz` the thicknesses. That is the entire scaffolding the solver sits on.

7 Step 2 — from calculus to arithmetic

7.1 Method: differences instead of derivatives

The heat equation is written in calculus (the ∂ symbols mean "rate of change"). A computer cannot do calculus, but it can do arithmetic on our layers. The trick (the **finite-difference** or **control-volume** method, Figure 3) replaces each rate of change with a simple difference between neighbours: the gradient between two layers $\approx$ (their temperature difference) $\div$ (distance between their centres); the heat stored in a layer $\approx \rho c_p \times$ (thickness) $\times$ (temperature change).

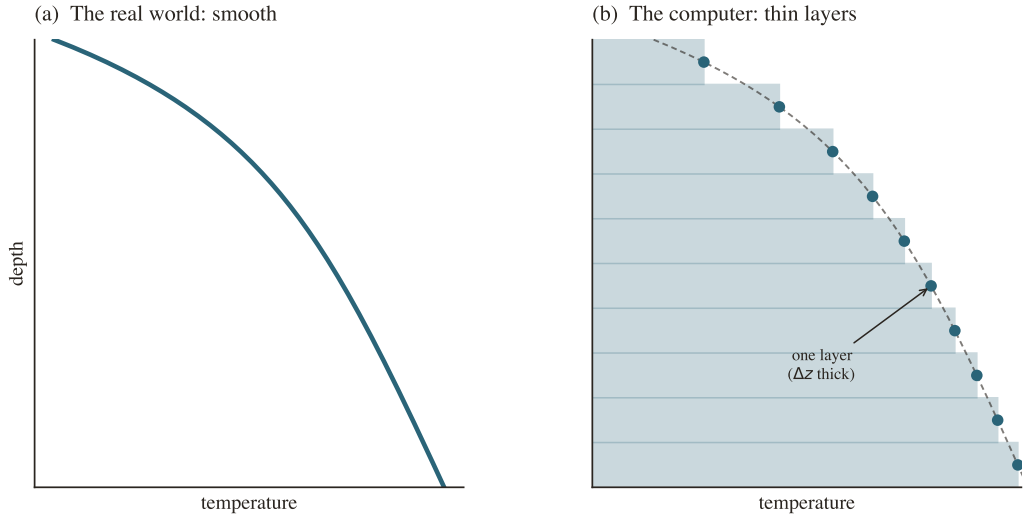


Figure 3. From smooth to computable. (a) Real temperature varies smoothly with depth. (b) The computer represents it as a stack of thin layers, each one number, updated step by step; fine enough layers reproduce the smooth curve (dashed).

7.2 Conductivity at the boundary between two layers

Heat passing from layer i to $i{+}1$ crosses the boundary between them, where the two layers have different conductivities. The physically correct combination is the **harmonic mean** — it correctly lets the worse insulator dominate, exactly like two resistors in series (the bigger resistor sets the flow):

`file: lunar/solver.py`

```
def _face_harmonic_mean(K):            # K = conductivity at each centre
    K_face = np.empty(K.size + 1)
    for i in range(1, K.size):
        K_face[i] = 2.0 * K[i-1]*K[i] / (K[i-1] + K[i])    # harmonic mean
    return K_face
```

7.3 Stepping forward in time: Crank–Nicolson

To advance temperature by one time step we use the **Crank–Nicolson** scheme: estimate the heat flows using the *average* of the situation now and one step ahead. Using only "now" (fully explicit) can blow up unless the steps are tiny; using only "next" (fully implicit) is stable but less accurate; averaging both is stable *and* accurate. This turns one time step into a set of linked equations — one per layer — where each layer's new temperature depends on its own and its two neighbours' new temperatures.

7.4 Code: assembling one time step

The solver builds those linked equations as four lists (`a`,`b`,`c`,`d`): `a` and `c` are each layer's coupling to the layers above and below, `b` is the layer itself, and `d` is the known right-hand side from the current temperatures.

file: lunar/solver.py (the heart of _step, lightly trimmed)

```
K   = K_func(T_prev, grid.z_mid)         # conductivity at every layer, now
rho = rho_func(grid.z_mid)               # density at every layer
cp  = cp_func(T_prev)                    # specific heat at every layer
cap = rho * cp * dz                      # heat capacity of each layer
K_face = _face_harmonic_mean(K)          # conductivity at the boundaries

# coupling of each layer to its neighbours (how easily heat crosses):
alpha_l[i] = dt * K_face[i]   / (dz_c[i]   * cap[i])   # to layer above
alpha_r[i] = dt * K_face[i+1] / (dz_c[i+1] * cap[i])   # to layer below

# the linked equation for each layer (Crank-Nicolson, theta = 0.5):
a[i] = -0.5 * alpha_l[i]                          # neighbour above
c[i] = -0.5 * alpha_r[i]                          # neighbour below
b[i] =  1.0 + 0.5*(alpha_l[i] + alpha_r[i])       # the layer itself
d[i] =  0.5*alpha_l[i]*left \
        + (1 - 0.5*(alpha_l[i]+alpha_r[i]))*T_prev[i] \
        + 0.5*alpha_r[i]*right                    # the known part
```

How the code does it

Read `alpha` as "how much heat can cross between these layers in one step, relative to how much the layer can hold" — big `alpha` means strongly coupled. The `a,b,c` lines hold the unknown new temperatures (the `0.5` is the Crank–Nicolson averaging); `d` packages everything already known from the current temperatures `T_prev`. Doing this for every layer gives a system of linked equations to solve — next.

8 Step 3 — solving the layer equations fast

Because each layer touches only its two immediate neighbours, the linked equations form a special, easy shape called **tridiagonal** (only the diagonal and its two neighbours are non-zero). There is a classic, very fast recipe — the **Thomas algorithm** — that sweeps down once and back up once, instead of the slow general method. For hundreds of layers, every hour of a multi-year simulation, this speed matters.

file: lunar/solver.py

```
def _thomas(a, b, c, d):            # solve the tridiagonal system A x = d
    n = b.size
    cp = np.empty(n); dp = np.empty(n); x = np.empty(n)
    cp[0] = c[0]/b[0];  dp[0] = d[0]/b[0]
    for i in range(1, n):                       # forward sweep
        m = b[i] - a[i]*cp[i-1]
        cp[i] = c[i]/m if i < n-1 else 0.0
        dp[i] = (d[i] - a[i]*dp[i-1]) / m
    x[n-1] = dp[n-1]
    for i in range(n-2, -1, -1):                # back substitution
        x[i] = dp[i] - cp[i]*x[i+1]
    return x
```

How the code does it

The first loop sweeps surface-to-bottom, eliminating each layer's dependence on the one above (`cp`,`dp` are cleaned-up coefficients). The second loop sweeps back up, reading off each layer's new temperature `x` from the one below. The output `x` is the temperature of every layer after this time step.

9 Step 4 — the surface: balancing sunlight and glow

9.1 Physics: the surface energy balance

The top layer is special: its temperature is whatever makes energy balance at every instant — sunlight absorbed must equal infrared light radiated away *plus* heat conducted downward into the soil:

The equation, term by term

$$\underbrace{(1-A)\,F_\odot}_{\text{sunlight absorbed}} \;-\; \underbrace{\varepsilon\,\sigma\,T_s^4}_{\text{infrared glow out}} \;-\; \underbrace{K\,\frac{T_s - T_{\text{below}}}{\Delta z/2}}_{\text{conducted downward}} \;=\; 0$$

A albedo (fraction of sunlight reflected, so $1-A$ is absorbed, ≈ 0.87 here); $F_\odot$ incoming sunlight; ε emissivity (≈ 0.95); σ the Stefan–Boltzmann constant; T_s the surface temperature we solve for. The glow grows as T_s^4 — very steeply — which is why this cannot be solved by simple algebra.

At lunar noon this balance gives $T_s \approx 390\,\text{K}$ (a scorching 117 °C); deep in the night, with no sunlight, it drops below 100 K.

9.2 Method: Newton's method finds T_s

Because T_s appears as T_s^4, there is no tidy formula. So the code *guesses, checks the imbalance, and corrects* — repeatedly, using **Newton's method**, which uses the slope of the imbalance to make each correction. It converges in a handful of tries.

file: lunar/solver.py

```
def surface_energy_balance_residual(T_s, insolation, albedo, emissivity,
                                    K_surf, dz_surf, T_subsurf):
    radiative_in  = (1.0 - albedo) * insolation          # sunlight absorbed
    radiative_out = emissivity * SIGMA_SB * T_s**4        # infrared glow
    conductive    = K_surf * (T_s - T_subsurf)/(0.5*dz_surf)  # into soil
    return radiative_in - radiative_out - conductive     # the imbalance
```

file: lunar/solver.py (Newton iteration, trimmed)

```
for _ in range(max_iter):
    R  = surface_energy_balance_residual(T_s, ...)       # current imbalance
    dR = -4.0*emissivity*SIGMA_SB*T_s**3 - 2.0*K_surf/dz_surf  # its slope
    T_s_new = T_s - R/dR                                 # Newton correction
    if abs(T_s_new - T_s) < tol:
        return T_s_new                                   # converged
    T_s = T_s_new
```

How the code does it

The first function returns the energy imbalance `R` for a trial surface temperature; zero means balanced. Newton's update `T_s_new = T_s - R/dR` nudges the guess toward balance using the imbalance's slope `dR`; when the nudge becomes tiny, the surface temperature for this instant is found.

10 Step 5 — the bottom: the geothermal flux

At the bottom we impose the steady interior heat: a fixed upward flux Q_b. This is what makes the deep temperature climb with depth at all — the column is fed from below, not sealed off (sealing it, a zero-flux bottom, was a conceptual error this project explicitly avoids). In the time-step assembly the bottom layer simply receives an extra dollop of heat each step:

file: lunar/solver.py

```
d[-1] += dt * inputs.Q_b / cap[-1]     # steady heat into the bottom layer
```

How the code does it

`d[-1]` is the bottom layer's known right-hand side; adding `dt*Q_b/cap[-1]` injects the geothermal heat (Q_b over a time `dt`, divided by the layer's heat capacity) every step. That one line is the entire bottom boundary condition.

11 Step 6 — settling to equilibrium

11.1 Physics: why settling is slow

With the recipe above the computer can march the lunar day forward step by step. But to get the *settled* repeating state, it must run until the column forgets wherever it started. The time to forget scales as $\tau \sim L^2/\kappa$ (depth squared, over diffusivity): the surface skin ($L \sim$ a few cm) forgets in a few lunar days, but the full 5 m column takes the equivalent of *roughly a thousand* lunar days. Marching that long for every trial K_d would be hopelessly slow.

11.2 Method and code: the shortcut

The fix uses the steady-state fact from Step 5: once settled, the same heat must flow through every layer, which fixes the entire deep profile directly — no waiting. The **equilibrium solver** alternates a short march (to settle the fast skin) with a direct reconstruction of the deep part, repeating until nothing changes.

`file: lunar/equilibrium.py (the outer loop, trimmed)`

```
for it in range(1, max_outer + 1):
    out = solve_pixel(... n_lunations_spinup=n_inner ...)  # short march
    T_mean = out.T.mean(axis=1)                            # cycle-average
    anchor = T_mean[i0]                                    # temp at 0.55 m
    drift  = abs(anchor - anchor_prev)
    # rebuild the deep profile from steady-state heat balance:
    T_init = _reconstruct_subskin(T_mean, z, i0, Q_b, K_func, u_rect)
    if drift < anchor_tol_K and it >= 2:
        break                                              # converged
    anchor_prev = anchor
```

How the code does it

Each pass does a short simulation (`solve_pixel`), reads the settled temperature at an anchor depth (0.55 m), then rebuilds the deep profile directly from the heat-balance law (`_reconstruct_subskin`). When the anchor stops moving between passes (`drift < anchor_tol_K`), the true settled state has been reached — in a handful of passes, at the cost of the old short run rather than a thousand-day march.

Key takeaway

Solving the model = grid (Step 1) + time-stepping each layer (Steps 2–3) + the two boundary rules (Steps 4–5) + settling to equilibrium (Step 6). Those pieces are the entire forward model. Why settling matters so much gets its own section (§19).

12 A worked trace: watching the machine run

12.1 One time step, a layer at 0.5 m, around 252 K

The code calls `conductivity_hayne(252, 0.5, Kd=4.58e-3)`. The depth bracket is essentially K_d (well below $H = 6$ cm); the radiative bracket is $1 + 2.7 \times (252/350)^3 \approx 2.0$; so $K \approx 9.2 \times 10^{-3}$. Density is $\rho \approx 1800$ and $c_p \approx 670$, so the heat capacity `cap` sets how far the temperature moves. The solver compares the heat arriving from the slightly cooler layer above and the slightly warmer layer below, and nudges this layer by a tiny fraction of a degree. Repeat for every layer, every hour of the lunar day, for the whole simulation.

12.2 One full retrieval, Apollo 15

The pipeline tries many K_d values; for each it runs the equilibrium solver and scores the predicted deep temperatures against the 7 real sensors with the RMSE:

trial K_d (mW m^{-1} K^{-1})	RMSE (K)
3.4 (the global value)	1.03
4.58 (the best fit)	**0.95**
much higher	rises again

The RMSE is lowest at $K_d = 4.58$ — the bottom of the bowl in Figure 6b — so that is Apollo 15's retrieved conductivity.

PART III • From Data to a Number, and its Uncertainty

13 The Apollo data and how it is cleaned

On Apollo 15 (1971) and 17 (1972), astronauts lowered thermometers more than two metres down (Figure 4); the records were restored from the original 1970s tapes in 2018. Two honest housekeeping steps turn the raw multi-year wiggling records into clean numbers.

1. The borestem cut. The fibreglass stem that holds the thermometers leaks a little surface heat downward, contaminating the shallowest sensors. Rather than guess a correction, we discard *every* sensor above 80 cm before looking at any answer.

2. Stability windows. Each record starts with disturbances (the drilling, equipment glitches, slow drift) and only later settles. For each sensor the code automatically scans for the longest flat stretch at the end — the part whose temperature trend is statistically indistinguishable from a flat line (a slope under 0.08 K per year) — and averages it into one "equilibrium" temperature. That single number per sensor is what the model is compared against.

After cleaning: 7 trustworthy points at Apollo 15, 16 at Apollo 17 — 23 numbers in total. The whole study extracts the most defensible conclusion from those 23 numbers.

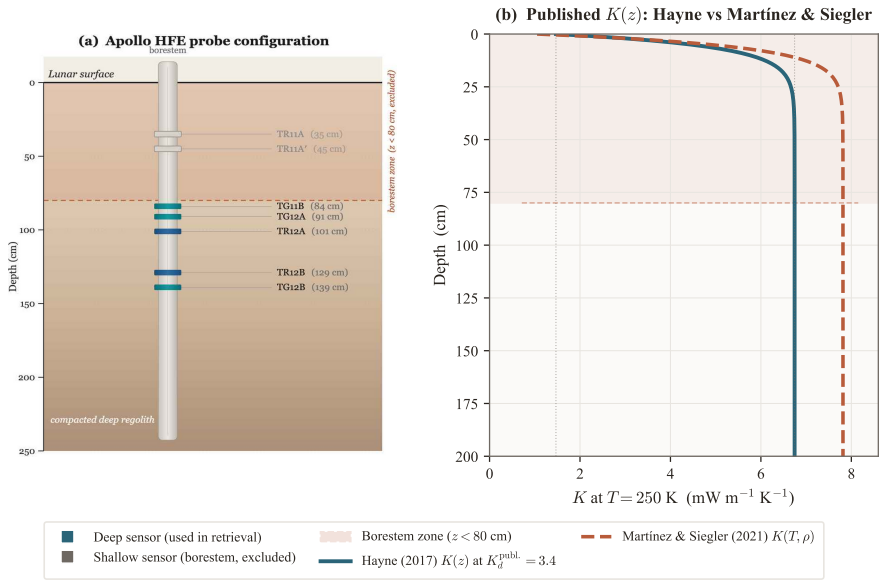


Figure 4. The probe. A fibreglass stem with thermometers at several depths. Shaded band ($z < 80$ cm): discarded. Right panel: the two textbook conductivity models compared later.

14 The retrieval: guess, simulate, compare

We cannot measure K_d directly, so we reverse the problem (Figure 6a): guess a K_d, simulate the temperatures it produces, score the mismatch, keep the best.

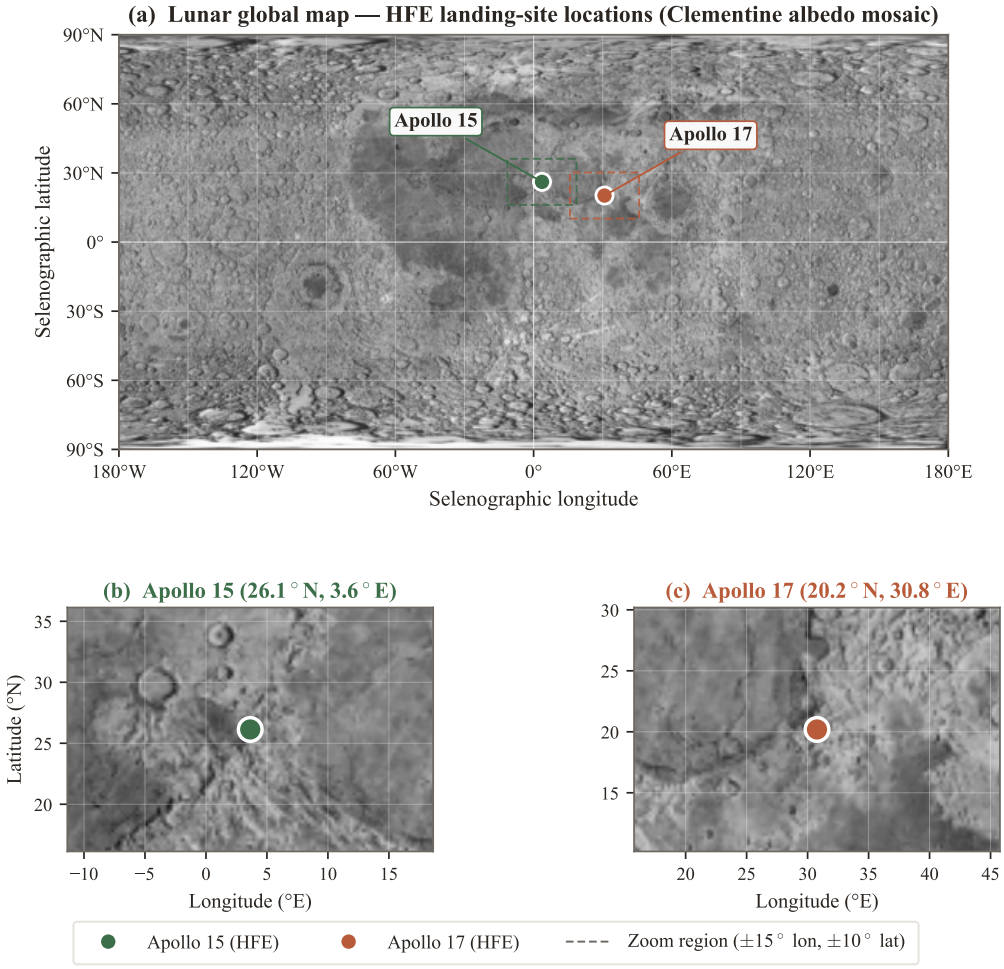


Figure 5. Where the boreholes are. Apollo 15 on a mare/highland boundary; Apollo 17 in a lava-filled valley — different geology, a physical reason the two sites could differ.

The equation, term by term

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(T_i^{\mathrm{model}} - T_i^{\mathrm{data}}\right)^2}$$

For each of N sensors take the model−data gap, square it (big misses count more, sign ignored), average, square-root back to degrees. Zero is perfect. The lowest-RMSE K_d is the answer, K_d^*.

Key takeaway

The retrieved conductivity is the bottom of the mismatch bowl: 4.6 at Apollo 15, 8.1 at Apollo 17 — both above the global 3.4, and different from each other.

15 Uncertainty I: the bootstrap

With only 7 or 16 points, how sure are we? We cannot redo Apollo, but we can *resample* our own data: treat the sensors as a bag of tickets and draw a new set *with replacement* — some sensors picked twice, some left out (Figure 7). Each fake set gives a slightly different K_d; doing this 1,500 times, the spread of answers is the uncertainty.

In plain words

Shuffling your own sample mimics the spread you would get from repeating the whole experiment — letting a tiny dataset honestly report its own reliability, with no assumptions about bell curves.

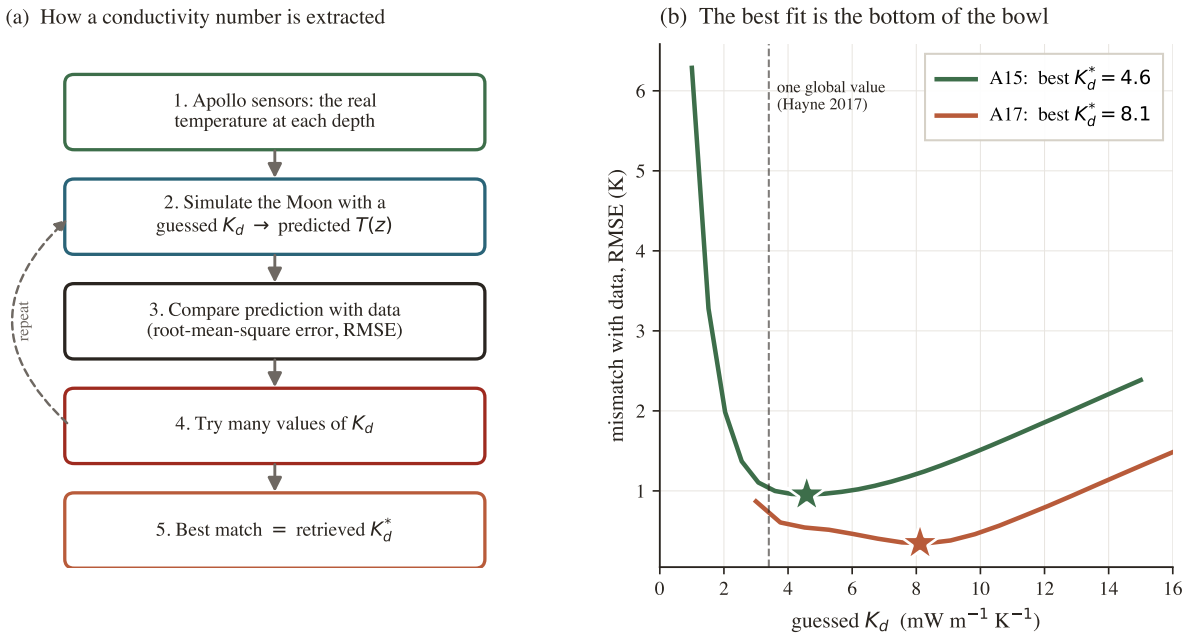


Figure 6. Extracting a number. (a) The guess–simulate–compare loop. (b) The real result: mismatch (RMSE) versus guessed K_d forms a bowl whose bottom is the best fit — Apollo 15 near 4.6, Apollo 17 near 8.1; dashed line is the single global value (3.4).

Key takeaway

Bootstrap = measure uncertainty by reshuffling your own data many times and watching the answer jump. Small jumps mean confident. The two sites' difference stays positive in ∼99% of reshuffles — that is what "statistically significant" means here.

16 Uncertainty II: Bayesian inference and MCMC

A complementary method lets us fold in prior knowledge of the heat flux Q_b. **Bayesian inference** combines the **prior** (what we believed before) and the **likelihood** (what the data say) into the **posterior** (updated belief), sharper than either (Figure 8a).

The equation, term by term

$$\underbrace{P(\text{answer} \mid \text{data})}_{\text{posterior}} \propto \underbrace{P(\text{data} \mid \text{answer})}_{\text{likelihood}} \times \underbrace{P(\text{answer})}_{\text{prior}}$$

Read $P(A \mid B)$ as "how plausible A is, given B." The posterior is the likelihood times the prior; "$\propto$" means we only need the shape, then normalise.

With two unknowns (K_d and Q_b) the posterior is a landscape we cannot write down, so **Markov Chain Monte Carlo (MCMC)** explores it with random "walkers" that wander toward more-plausible spots, leaving footprints whose density maps the answer — no formula needed. It reveals a long diagonal **ridge** (Figure 8b): one borehole fixes only the gradient, i.e. the ratio Q_b/K_d — halve both and nothing changes. This is why the absolute K_d depends on the assumed Q_b, while the *difference* between sites is sturdier.

Key takeaway

Bayesian/MCMC = explore every answer consistent with data and prior knowledge using random walkers. For us it confirms the bootstrap and exposes the K_d–Q_b ridge.

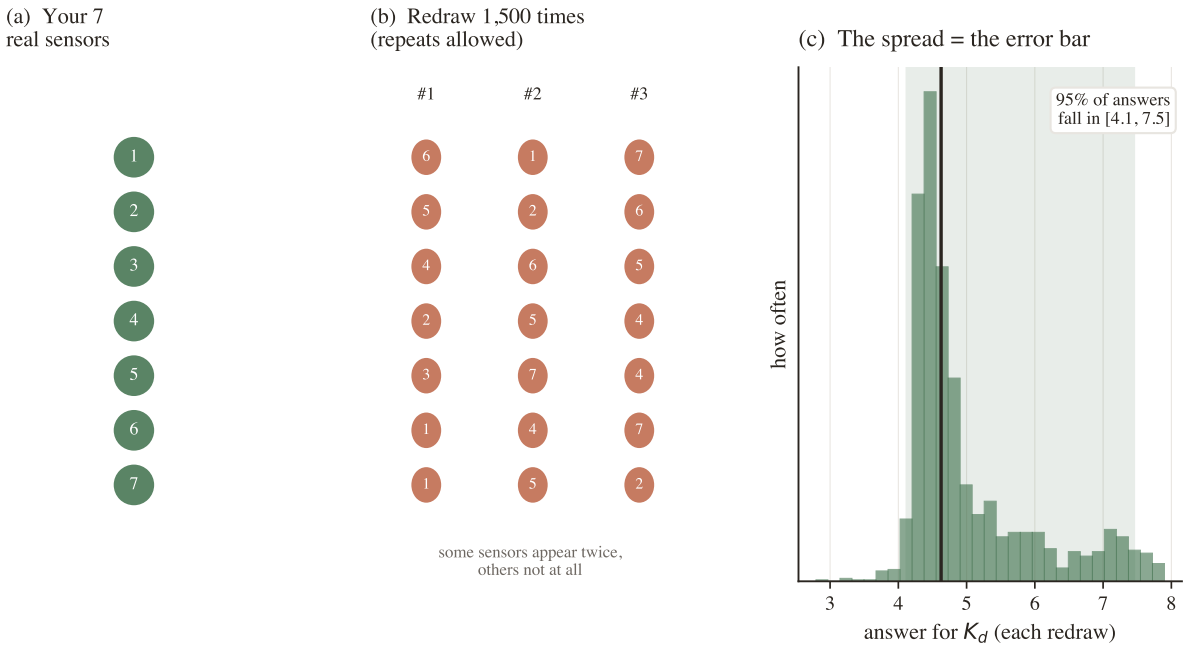


Figure 7. The bootstrap, from scratch. (a) Your real sensors. (b) Many resampled sets (repeats allowed, some left out); each yields its own K_d. (c) The spread of those answers is the error bar — 95% of Apollo 15's resamples fall in [4.1, 7.5].

17 Choosing between explanations: the AICc

Two stories compete: "different conductivity per site" versus "same conductivity, different heat flux." We cannot just pick whichever fits closer, because a model with more free knobs can *always* fit better, even by chasing noise — **overfitting** (Figure 9a). The **Akaike Information Criterion** ("c" for a small-sample correction) scores each model as its misfit *plus a penalty per free parameter*; lowest wins — good fit without needless complexity.

The equation, term by term

$$\mathrm{AICc} = \underbrace{N \ln(\text{misfit}/N)}_{\text{rewards a good fit}} + \underbrace{2k}_{\text{penalty per knob}} + \underbrace{\frac{2k(k+1)}{N-k-1}}_{\text{small-sample fix}}$$

N data points, k free parameters. Only differences between models matter: a gap $\gtrsim 2$ is meaningful, $\gtrsim 10$ decisive.

Scoring our three candidate models, the "different K_d per site" model (M1) wins; the "same K_d" alternatives score worse even though they were allowed to fit — the data prefer genuinely different conductivities.

Key takeaway

AICc = a scoreboard rewarding fit but fining complexity. It picks the per-site-conductivity explanation over one-size-fits-all.

18 The whole pipeline at a glance

The pieces above run in this order, each writing a file the next reads:

1. `lunar/` — the engine (properties, grid, solver, equilibrium), used by everything.
2. `scripts/pipeline/retrieve_kd.py` — runs the retrieval and bootstrap, writes `output/kd_retrieval_results.json`.
3. `scripts/pipeline/` (others) — the Bayesian MCMC, the AICc model comparison, and sensitivity checks, each writing its own JSON.
4. `scripts/figures/` — read those JSONs and draw every figure in the paper and in this book.

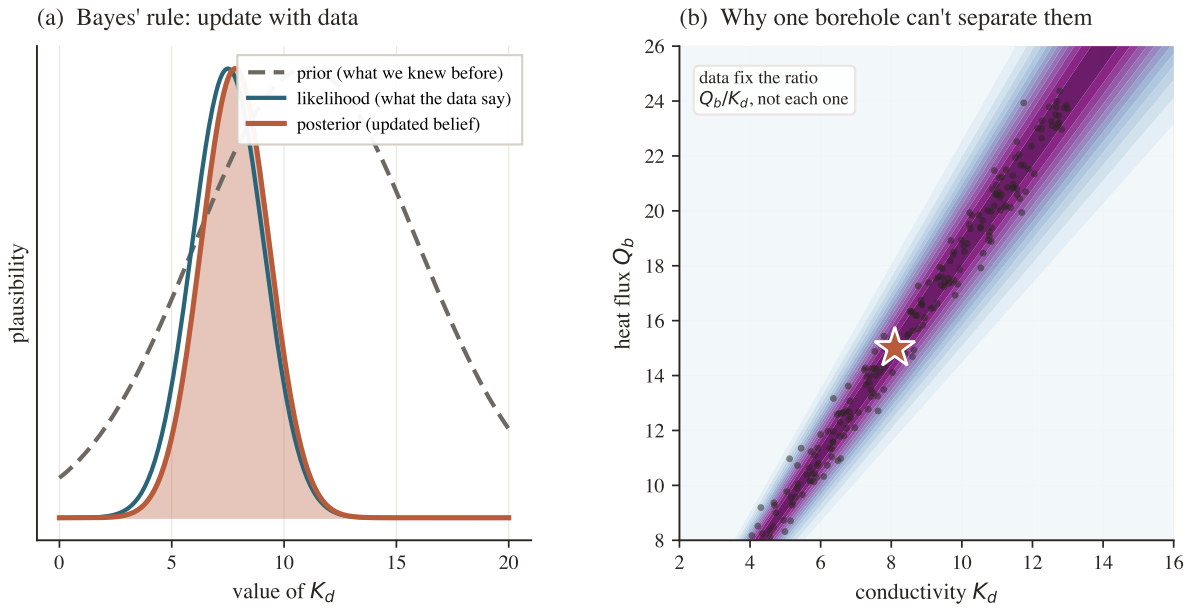
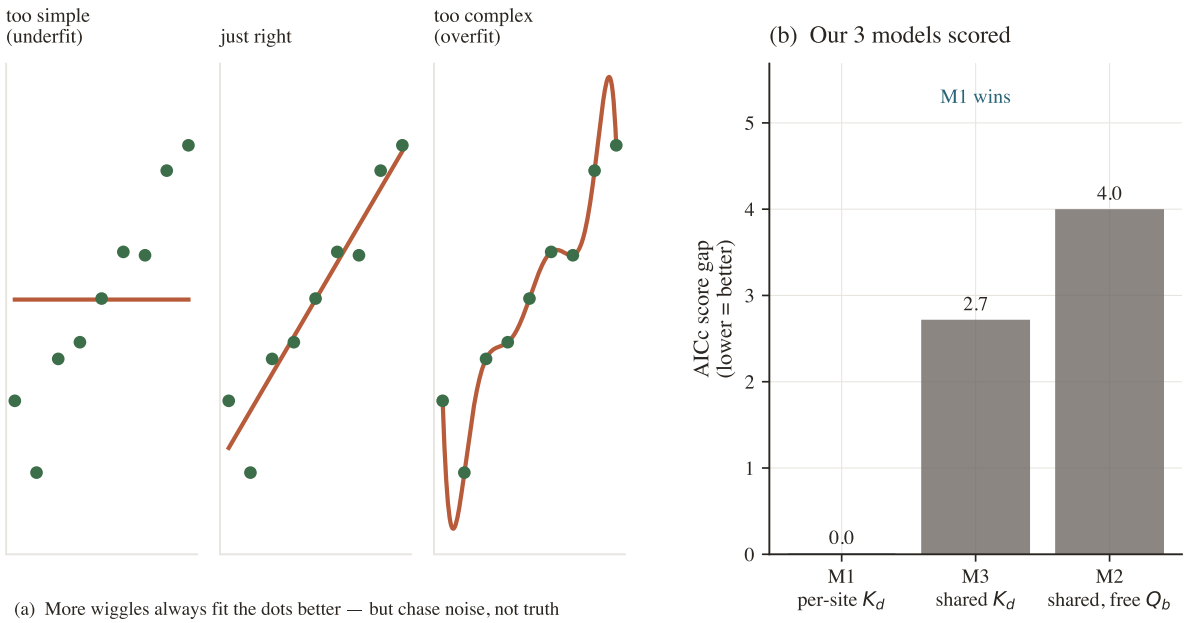


Figure 8. Bayesian updating and the ridge. (a) Prior × likelihood = posterior. (b) Our 2-D landscape: plausibility is high all along a diagonal *ridge* because the data fix the *ratio* Q_b/K_d, not each separately. Dots are MCMC "footprints"; the star is the published value.

Because every result is a committed file, anyone can verify the paper's numbers without rerunning a thing.



(a) More wiggles always fit the dots better — but chase noise, not truth

Figure 9. Overfitting and the AICc. (a) Too simple misses the trend; too complex chases noise; the middle is right. (b) The AICc adds a penalty for extra knobs and scores our three real models: M1 (per-site K_d) wins.

PART IV • The Result

19 The mistake we found and fixed

19.1 The bug

To start a simulation the computer needs an initial deep temperature. The old code typed one in (250 K for Apollo 15, 255 K for Apollo 17) and ran only 30 lunar days — far short of the ∼1000 needed to settle. So the deep temperature was mostly the typed-in guess, and the conductivity read from it was partly an echo of that guess. We proved it (Figure 10a): three starting guesses gave three different deep profiles. An answer that moves with an arbitrary guess is not a measurement.

19.2 The fix and the check

The equilibrium solver of §11 computes the settled state directly. We verified it: starting from 240 vs 260 K now agrees to within 0.02 K, and a long check-run does not drift (Figure 10b). We did *not* just "run longer" — we used the heat-balance shortcut to jump to the answer.

19.3 What changed

Apollo 15: 4.86 → 4.58 — barely moved (its guess was close by luck). **Apollo 17:** 11.23 → 8.12 — dropped a lot (its guess was too warm and had inflated the value).

Key takeaway

The fix made the science *stronger*: the site difference became statistically significant, Apollo 17's value moved into the range other evidence expects, and an independent model that used to contradict it now agrees. When a fix makes independent checks line up, it is the right fix.

20 The results and what they mean

	Apollo 15	Apollo 17
Retrieved K_d^* (mW m^{-1} K^{-1})	4.58	8.12
95% confidence interval	[4.1, 7.5]	[6.9, 9.0]
Difference (17−15)	3.3, 95% CI [0.4, 4.6], significant	

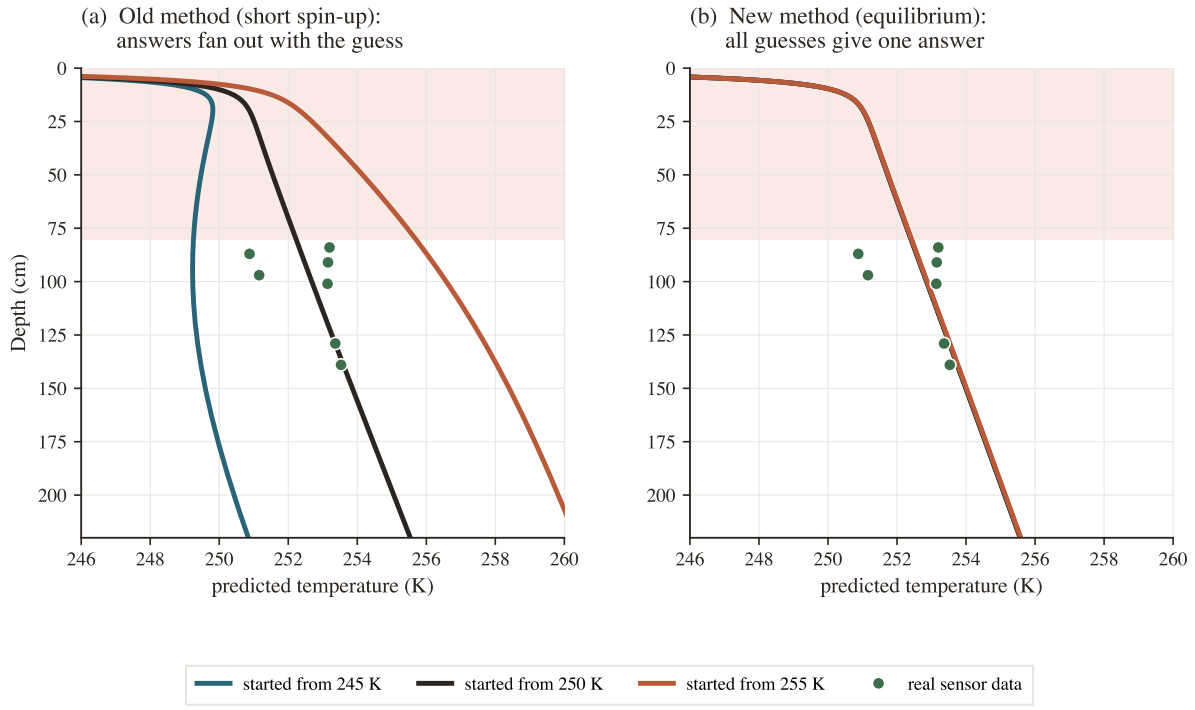


Figure 10. The bug and the fix (real runs, Apollo 15). (a) Old method: starting from 245/250/255 K, the deep profile fans out — the guess leaks in. (b) New method: all three land on one curve through the data. The answer no longer depends on the guess.

For scale: water ∼600, styrofoam ∼30, lunar regolith near 5–8 in these units — an exceptional insulator. The fitted profiles are in Figure 11.

Why these values make sense. Apollo 15's 4.6 sits between the global 3.4 and orbital ∼7, matching soil porosity. Apollo 17 being ∼1.8× higher follows from two effects pointing the same way: its soil is volcanic basalt (more conductive than Apollo 15's highland rock) and slightly more packed — neither pushed to an extreme.

An independent sanity check. Using only *deep* temperatures for the fit, we then checked the models against the *surface* swing seen from orbit (Diviner). They match (Figure 12), confirming the whole forcing chain is right — a test that used no data from the retrieval.

21 Honest limits and the bottom line

1. **Absolute values lean on Q_b.** The data fix the ratio Q_b/K_d (the ridge), so revising the heat flux shifts K_d proportionally. The site *difference* is far more robust.
2. **Only two boreholes exist.** Two points cannot map the whole Moon — only show these two differ. The rest needs new missions.
3. **Apollo 17 is systematics-sensitive.** Its error bar is tight, but its accuracy depends on a few inputs, reported openly in the paper's error budget.

Key takeaway

Rock-solid: the two sites need different conductivities, Apollo 17 the more conductive. **Conditional:** the exact numbers, which depend on the assumed heat flux and model form. The paper says, clearly, which is which — and that honesty is what makes it credible.

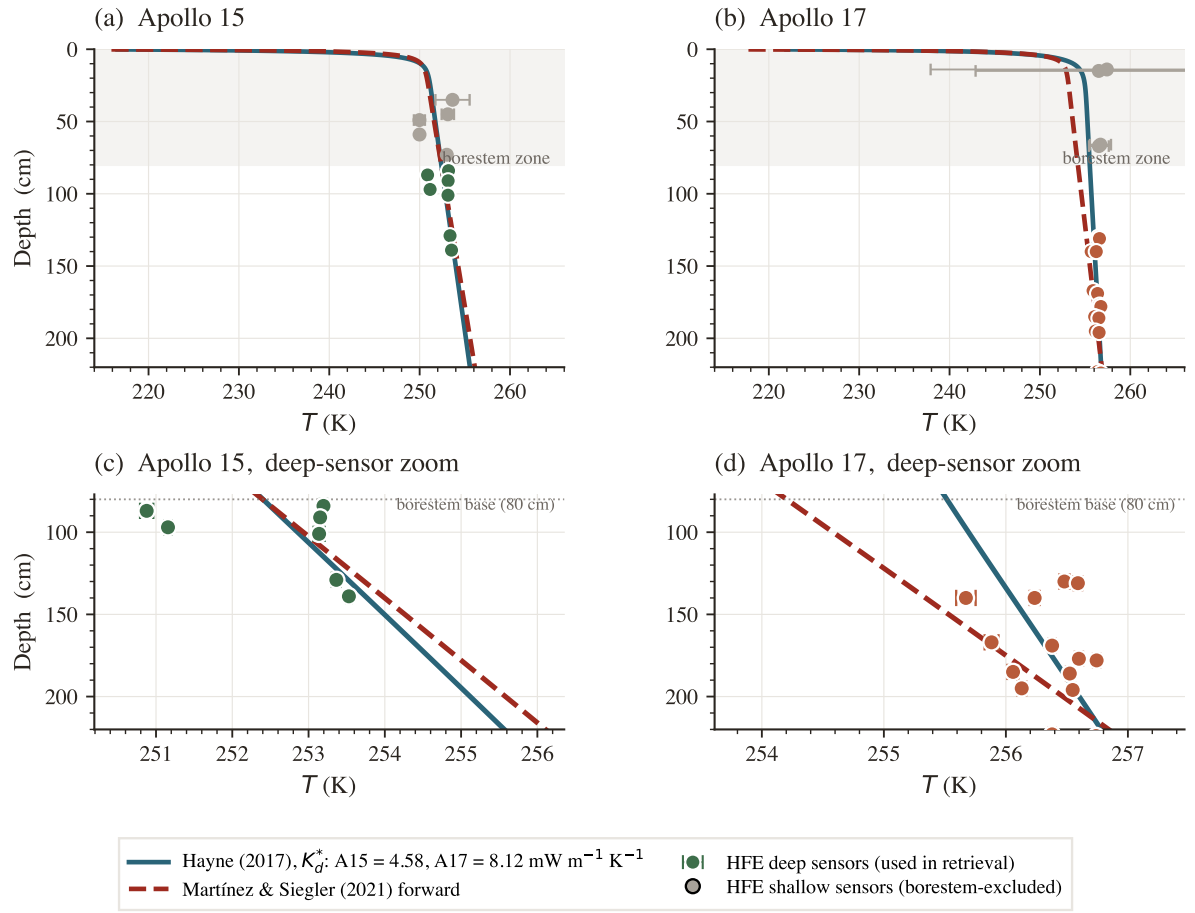


Figure 11. The fitted profiles. Dots: real deep sensors; curves: best-fit models. Apollo 15 rises *more steeply* (lower conductivity needs a steeper gradient to pass its heat); Apollo 17, more conductive, rises gently.

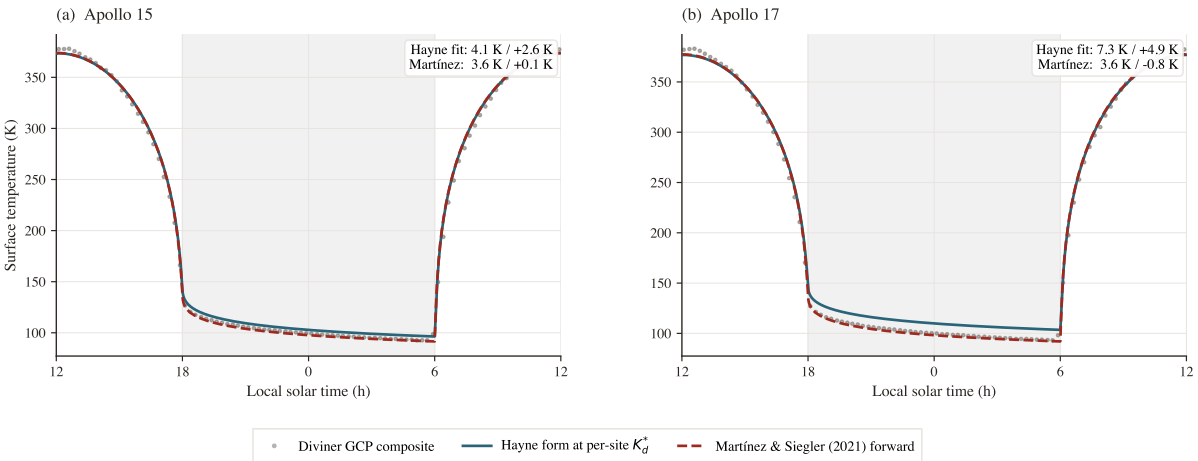


Figure 12. Independent check. Grey dots: surface temperature over a lunar day from orbit. Curves: our models, tuned only to the deep sensors. The agreement confirms the setup.

PART V • Appendices

22 How to run the code yourself

```
git clone https://github.com/rp3gregorio/Lunar-HFE.git
cd Lunar-HFE
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

python scripts/pipeline/retrieve_kd.py          # the core retrieval
python scripts/figures/make_letter_figures.py   # the figures
python scripts/figures/make_results_figures.py
```

The pieces, mapped to this book:

- `lunar/properties.py` — material formulas (§3)
- `lunar/grid.py` — the depth grid (§6)
- `lunar/solver.py` — time-stepping, tridiagonal solve, both boundaries (§7–11)
- `lunar/equilibrium.py` — the settling solver (§12, §19)
- `lunar/constants.py` — every number, with its source
- `scripts/pipeline/` — retrieval, bootstrap, MCMC, AICc
- `output/` — committed results, to verify every number
- `docs/FLAG_REPORT.md` — the audit behind §19

23 Glossary of every term

Conduction
Heat spreading through solid material, atom to atom.

Temperature / heat
Average atomic jiggle / the energy of jiggling on the move.

Conductivity (K, K_d)
How easily heat passes; K_d is the deep settled value — our target. Small = good insulator.

Heat flux (q, Q_b)
Heat crossing one m^2 per second; Q_b is the steady flux from the Moon's interior.

Gradient
Temperature change per metre of depth.

Fourier's law
Flux = conductivity × gradient.

Heat equation
Temperature changes where more heat flows in than out.

Density (ρ) / specific heat (c_p)
Mass per volume / energy to warm 1 kg by 1 K.

Albedo (A) / emissivity (ε)
Fraction of sunlight reflected / efficiency of infrared glow.

Skin depth
How far the day–night swing reaches (a few cm).

Steady state
Settled condition, unchanging day to day.

Boundary conditions
Rules at the column ends: surface radiation balance; fixed flux Q_b at the bottom.

Grid / discretisation
Splitting the column into layers and time into steps for the computer.

Finite difference
Replacing a calculus rate-of-change with a difference between neighbours.

Crank–Nicolson
A stable time-step scheme averaging now and next.

Tridiagonal / Thomas
The simple shape of the layer equations, and the fast recipe that solves them.

Harmonic mean
The correct way to average conductivity at a layer boundary (worse insulator dominates).

Newton's method
Guess-and-correct using slope, for the surface temperature.

Borestem cut
Discarding sensors above 80 cm (stem contamination).

Stability window
The settled tail of a sensor's record that is averaged into one temperature.

Retrieval
Guessing K_d, simulating, keeping the best match.

RMSE
One number for the typical model–data gap, in degrees.

Bootstrap
Resampling your own data to measure uncertainty.

Bayesian / prior / likelihood / posterior
Updating belief: before / data / after.

MCMC
Random walkers mapping all plausible answers.

Degeneracy / ridge
When data fix only a combination (Q_b/K_d), not each unknown.

Overfitting / AICc
Fitting noise / a score that fines complexity.

HFE, A15/A17
Heat-Flow Experiment; the Apollo 15 and 17 sites.